

Designing a Production-Ready Backtesting Engine for Swing-Trading Portfolios

Overview

Swing-trading in equity markets involves holding positions for several days to a few weeks and requires a **robust backtesting engine** that can optimise a small portfolio (4–6 stocks) for **maximum risk-adjusted returns** and provide reliable metrics for live trading. This report provides both the theoretical foundations and a practical blueprint for implementing such a backtesting system. It integrates best practices from institutional-grade backtesting, emphasises risk-adjusted performance, and offers code-level guidance for a Python implementation that can feed a Streamlit dashboard.

1 Theoretical Foundations

1.1 Return Calculations and Portfolio Mathematics

Quantity	Formula	Explanation
Simple return	$R_t = P_t - P_{t-1} / P_{t-1}$	Measures price change between consecutive periods.
Log return	$r_t = \ln(P_t / P_{t-1})$	Adds continuously compounded returns; sum of log returns approximates the portfolio's total return.
Expected return	$\mu = \frac{1}{T} \sum_{t=1}^T R_t$	Mean of historical returns.
Variance	$\sigma^2 = \frac{1}{T-1} \sum_{t=1}^T (R_t - \mu)^2$	Measures return variability.
Annualised volatility	$\sigma_{\text{ann}} = \sigma \sqrt{252}$	Scales daily volatility to an annual figure.
Covariance matrix	$\Sigma_{ij} = \text{Cov}(R_i, R_j)$	Covariance between asset i and j used for diversification and risk estimation.
Portfolio return	$R_p = w^\top R$	Weighted sum of individual returns where w is the weight vector.
Portfolio variance	$\sigma_p^2 = w^\top \Sigma w$	Variance of the weighted portfolio 【684837841290981†L80-L116】.
Sharpe ratio	$S = \frac{R_p - R_f}{\sigma_p}$	Measures excess return per unit of risk

Quantity	Formula	Explanation
		【288494383251952†L313-L326】. Higher values indicate better risk-adjusted performance.
Sortino ratio	$S_{\text{sortino}} = \frac{R_p - R_f}{\sigma_d}$	Uses downside deviation (σ_d) instead of total volatility to penalise downside risk 【512402959581916†L57-L76】.
Maximum drawdown	$MDD = \max \left(\frac{P_{\text{peak}} - P_{\text{trough}}}{P_{\text{peak}}} \right)$	Measures the largest decline from peak to trough 【354485727793711†L410-L425】.
CAPM regression	$R_{p,t} - R_f = \alpha + \beta (R_{m,t} - R_f) + \varepsilon_t$	Decomposes excess portfolio return into alpha and beta 【190274236457581†L18-L33】. Beta is computed as $\beta = \frac{\text{Cov}(R_p, R_m)}{\text{Var}(R_m)}$ 【591785665461704†L105-L116】. Alpha measures performance beyond the market risk premium.
Information ratio	$IR = \frac{R_p - R_b}{\sigma_{\text{tracking}}}$	Compares the portfolio's active return to its tracking error; tracking error is the standard deviation of $(R_p - R_b)$ 【158632977684089†L116-L155】 【4598424213241†L120-L170】.
Turnover	$\text{Turnover}_t = 1/2 \sum_i w_{i,t} - w_{i,t-1} $	Fraction of portfolio traded each period; half the sum of absolute changes in weights 【631569097049119†L209-L215】.
Value at Risk (VaR)	$VaR_\alpha = (\mu - z_\alpha \sigma) V$	Variance-covariance VaR uses z_α (z-score at confidence level α), portfolio mean μ , volatility σ , and portfolio value V

Quantity	Formula	Explanation
Conditional VaR (CVaR/ES)	$CVaR_{\alpha} = E[L \mid L \geq VaR_{\alpha}]$	Expected loss given that the loss exceeds VaR; for historical simulation it is the average of the worst α % losses 【219790666543552†L139-L152】 .
CAGR	$CAGR = (V_{\text{final}}/V_{\text{initial}})^{1/T} - 1$	Compound annual growth rate measures annualised return 【472859432363745†L124-L146】 . 【640443988010342†L53-L69】 .

1.2 Best Practices for Institutional-Grade Backtesting

1. Data integrity and bias prevention

- *Survivorship bias*: restrict the universe to stocks that existed throughout the backtest or use historical constituent lists. Survivorship bias skews results by excluding delisted or bankrupt stocks 【556435430300884†L48-L150】 .
- *Look-ahead bias*: ensure that only information available at the decision time is used. Avoid using future data for training or selecting parameters 【556435430300884†L48-L150】 .
- *Overfitting and data-snooping*: avoid excessive parameter tuning on historical data; apply **walk-forward optimisation** and out-of-sample validation. Use multiple evaluation periods to reduce the chance that results are due to chance 【830181354955615†L117-L133】 【830181354955615†L161-L182】 .
- *Transaction costs and slippage*: ignoring trading costs produces unrealistic performance 【556435430300884†L48-L150】 . Model brokerage, taxes, stamp duty and market impact. Use realistic cost estimates from Indian markets (see §2.3).
- *Period selection bias*: avoid cherry-picking a favourable backtest period; test across different market regimes 【556435430300884†L48-L150】 . Include both bull and bear phases and adjust for non-stationary conditions.

2. Walk-forward optimisation theory

Walk-forward optimisation divides data into **training** (in-sample) and **testing** (out-of-sample) windows. Parameters (e.g., portfolio weights) are optimised on the training window, then applied to the subsequent out-of-sample segment. The window is rolled forward, and the process repeats across multiple periods 【830181354955615†L117-

L133】 . This procedure mitigates overfitting and ensures that parameter estimates are evaluated out-of-sample. The sequence of training windows can overlap or be anchored; the key is consistent re-estimation and evaluation. A final evaluation across all out-of-sample windows provides an estimate of strategy robustness
【830181354955615†L161-L182】 .

3. Risk-adjusted performance metrics

- **Sharpe ratio:** widely used measure of excess return per unit of volatility. A ratio above 1 is generally considered acceptable; above 2 is strong and above 3 is excellent 【288494383251952†L336-L347】 . However, it assumes normally distributed returns and penalises upside and downside volatility equally.
- **Sortino ratio:** similar to Sharpe but uses downside deviation to capture downside risk 【512402959581916†L57-L76】 . Useful when returns are asymmetric.
- **Information ratio:** evaluates excess return relative to a benchmark while considering tracking error 【158632977684089†L116-L155】
【4598424213241†L120-L170】 . A higher IR indicates consistent outperformance.
- **Max drawdown:** captures the worst peak-to-trough decline
【354485727793711†L410-L425】 . Lower drawdowns indicate better capital preservation.
- **Alpha and beta (CAPM):** separate skill (alpha) from market exposure (beta)
【190274236457581†L18-L33】 【591785665461704†L105-L116】 . Statistically significant positive alpha implies risk-adjusted outperformance.
- **VaR/CVaR:** quantify tail risk at a chosen confidence level
【219790666543552†L139-L152】 【472859432363745†L124-L146】 . CVaR is more conservative since it averages extreme losses.
- **Turnover and cost metrics:** track how often the portfolio trades to anticipate transaction costs; high turnover erodes returns 【631569097049119†L209-L215】 .

1.3 Regime-Based Performance Analysis

Markets exhibit phases with different volatility and trend characteristics. Distinguishing **market regimes** improves strategy robustness and helps determine whether to deploy or reduce risk.

Volatility-based regimes

- **Normalized volatility measures:** The ATR divided by price ($ATR \div price$) and realised volatility computed from recent returns are practical measures to classify regimes
【953374386546441†L299-L316】 .

- **Contraction vs expansion:** Low-volatility periods often precede volatility expansion; monitoring rolling volatility identifies when calm periods are tightening or breaking open 【953374386546441†L299-L316】 .
- **Regime filter:** A risk filter can block trades during high-volatility regimes (e.g., when realised volatility or ATR exceeds a threshold). A Hidden Markov Model (HMM) can also classify hidden states into low-volatility and high-volatility regimes using historical returns 【654737399976859†L68-L81】 . QSTrader demonstrates training an HMM on past index returns and using predicted state probabilities to allow or cancel trades 【654737399976859†L105-L116】 .

Trend-based regimes

- **Moving-average crossovers:** Combining a short-term and long-term moving average identifies trend regimes. Fidelity notes that the **50-day and 200-day exponential moving averages (EMAs)** are commonly used; a **buy signal** (often called a *golden cross*) occurs when the shorter average crosses above the longer one, while a **sell signal** (death cross) occurs when the short average crosses below 【868227748718115†L447-L487】 . The 200-day EMA smooths long-term trends, whereas the 50-day EMA tracks recent price action 【868227748718115†L447-L456】 . Price above a moving average can act as support; price below acts as resistance 【868227748718115†L461-L470】 .
- **Trend strength and volatility matrix:** A 2×2 regime map (strong trend vs range crossed with low vs high volatility) guides trade permissions 【953374386546441†L359-L367】 . Trend-following playbooks should only be executed in strong-trend/low-volatility regimes, whereas mean-reversion strategies are more appropriate in ranges with non-elevated volatility 【953374386546441†L359-L367】 .

1.4 Common Pitfalls and Biases

1. **Overfitting:** Fitting models too closely to historical data yields poor generalisation. Use walk-forward optimisation, cross-validation, and penalise complexity. Monitor out-of-sample performance to ensure stability.
2. **Data-snooping:** Reusing data for multiple hypothesis tests increases false discoveries. Limit the number of parameter combinations tested and adjust for multiple comparisons.
3. **Survivorship and look-ahead bias:** Always use historical index constituents and avoid using future information. Align data by announcement date for fundamentals or corporate actions.
4. **Ignoring costs and liquidity:** Trading costs (brokerage, taxes, slippage) materially affect net returns 【556435430300884†L48-L150】 . Implement a realistic cost model.
5. **Unrealistic assumptions:** Stationarity, IID returns and normality rarely hold. Use robust metrics, heavy-tail risk measures (CVaR), and regime-dependent analysis.

2 Practical Implementation Blueprint

The following sections describe a modular backtesting engine built in Python. The engine supports dynamic user-specified tickers, uses **PyPortfolioOpt** to compute maximum-Sharpe weights, and compares the strategy against the **NIFTY 50 (^NSEI)** and an equal-weight benchmark. The modules can be integrated into a Streamlit dashboard.

2.1 System Architecture

1. Data Module:

- DataFetcher: fetches historical OHLCV data using yfinance or another API. Downloads daily data for user-specified tickers and benchmark (^NSEI).
- DataCleaner: aligns data across tickers, forward-fills missing values, and computes returns (simple and log). It also computes volatility metrics (rolling standard deviation) and ATR for regime detection.
- FeatureStore: stores computed metrics (returns, volatility, covariances) for each lookback window.

2. Optimizer Module:

- MaxSharpeOptimizer: uses PyPortfolioOpt to estimate expected returns (e.g., mean historical returns) and the covariance matrix for a rolling window. Computes maximum-Sharpe weights subject to constraints (weights sum to 1 and can be long-only or long-short if allowed). Optionally incorporate weight bounds or sector constraints.
- WeightStorage: records portfolio weights for each rebalance date.

3. Backtester Module:

- WalkForwardEngine: implements walk-forward optimisation: slides a training window (e.g., 252 trading days) across the data, optimises weights, then applies them out-of-sample for a specified rebalance period (e.g., monthly). The engine updates weights and handles missing assets by dropping or substituting with cash. It multiplies portfolio weights by realised returns to compute equity curve.
- TransactionCostModel: applies brokerage, taxes and slippage to trades. For India, assume the following approximate costs based on ClearTax data:
 - **Brokerage**: discount brokers charge a flat ₹0–₹20 per trade; full-service brokers charge 0.01–0.5% of trade value **【660910205238419†L165-L176】** . For modelling, use a 0.05% brokerage fee on each buy or sell.
 - **Securities Transaction Tax (STT)**: 0.1% on buy and sell for delivery trades; 0.025% on the sell side for intraday **【660910205238419†L184-L195】** .
 - **Exchange transaction charges**: ~0.00325% on NSE **【660910205238419†L219-L225】** .
 - **Stamp duty**: ~0.015% for delivery trades **【660910205238419†L239-L246】** .
 - **GST**: 18% of brokerage and transaction fees **【660910205238419†L197-L204】** .The cost per transaction is the sum of these components multiplied by traded

value. Slippage can be modelled as a fixed bps (e.g., 2 bps). The **turnover** metric helps estimate total transaction costs: $\text{cost} = \text{turnover} \times \text{cost_rate}$.

4. **Regime Filter Module:**

- VolatilityRegimeDetector: computes a normalised ATR ($\text{ATR} \div \text{price}$) and realised volatility. Defines regimes using thresholds (e.g., top 30% volatility quantile = high-volatility). Optionally train a Hidden Markov Model to classify high vs low volatility states and apply a trade permission filter, as described by QuantStart [654737399976859†L105-L116] .
- TrendRegimeDetector: uses moving-average crossovers (e.g., 10-day vs 30-day SMA or 50-day vs 200-day EMA). A golden cross (short SMA > long SMA) signals a bullish regime; a death cross signals bearish conditions [868227748718115†L447-L487] . Filter trades depending on regime and strategy type.

5. **Benchmark Module:**

- EqualWeightBenchmark: equal-weights the selected stocks at rebalancing dates and computes its performance.
- IndexBenchmark: uses ^NSEI returns for comparison.

6. **Metrics Module:**

- PerformanceMetrics: computes cumulative returns, CAGR [640443988010342†L53-L69] , volatility, Sharpe/Sortino ratios [288494383251952†L313-L326] [512402959581916†L57-L76] , max drawdown [354485727793711†L410-L425] , VaR/CVaR [219790666543552†L139-L152] [472859432363745†L124-L146] , turnover [631569097049119†L209-L215] , alpha/beta (via CAPM regression using statsmodels), information ratio [158632977684089†L116-L155] and tracking error. Provide both point estimates and rolling versions (e.g., 6-month rolling Sharpe).
- RiskDecomposition: decomposes portfolio returns into market beta and idiosyncratic alpha using regression [190274236457581†L18-L33] [591785665461704†L105-L116] . P-values evaluate statistical significance.

7. **Validation & Decision Module:**

- StrategyValidator: applies rules to accept or reject strategy instances. Examples:
 - Sharpe ratio ≥ 1.0 (exceeding the “acceptable” threshold [288494383251952†L336-L347]).
 - Statistically significant alpha (p-value < 0.05).
 - Maximum drawdown below a defined threshold (e.g., -20%).
 - Information ratio above 0.5.
 - Turnover below 100% per period to contain costs.
- ReportGenerator: produces summaries, charts, and tables for dashboard display.

2.2 Data Pipeline Implementation

A. Fetching and Cleaning Data

```
import yfinance as yf
import pandas as pd

class DataFetcher:
    def __init__(self, tickers, start, end):
        self.tickers = tickers
        self.start = start
        self.end = end

    def download(self):
        data = yf.download(self.tickers, start=self.start, end=self.end, progress=False)
        return data['Adj Close'] # adjust for splits/dividends

class DataCleaner:
    def __init__(self, price_df):
        self.price_df = price_df

    def align_and_fill(self):
        df = self.price_df.copy().dropna(how='all')
        df = df.fillna(method='ffill').fillna(method='bfill')
        return df

    def compute_returns(self, log=False):
        df = self.align_and_fill()
        if log:
            return np.log(df / df.shift(1)).dropna()
        else:
            return df.pct_change().dropna()

    def compute_atr(self, window=14):
        # For regime detection: compute ATR (for each asset separately)
        highs = self.price_df['High']
        lows = self.price_df['Low']
        close = self.price_df['Close']
        tr = pd.DataFrame({
            'tr1': highs - lows,
            'tr2': abs(highs - close.shift(1)),
            'tr3': abs(lows - close.shift(1))
        }).max(axis=1)
```



```
atr = tr.rolling(window).mean()
return atr
```

Notes:

- yfinance pulls daily OHLCV data. Adjusted prices ensure corporate actions are accounted for.
- Forward/backward filling handles missing values; remove assets with insufficient data.
- ATR is computed using high/low/close series for each ticker.

B. Return and Covariance Computation

```
import numpy as np
from pypfopt import expected_returns, risk_models
```

```
class FeatureStore:
```

```
    def __init__(self, returns):
        self.returns = returns
```

```
    def compute_expected_returns(self):
        # annualised mean returns for each asset
        mu = expected_returns.mean_historical_return(self.returns)
        return mu
```

```
    def compute_covariance(self):
        # Sample covariance matrix using exponentially weighted or sample estimator
        cov = risk_models.sample_cov(self.returns)
        return cov
```

2.3 Walk-Forward Backtester Implementation

```
from pypfopt import EfficientFrontier
from statsmodels.api import OLS, add_constant
```

```
class WalkForwardEngine:
```

```
    def __init__(self, returns, rebalance_freq='M', lookback=252, cost_rate=0.002):
        self.returns = returns
        self.freq = rebalance_freq # 'M' for monthly, 'W' for weekly
        self.lookback = lookback
        self.cost_rate = cost_rate # approximate total transaction cost per unit turnover
        self.weights_history = []
        self.portfolio_returns = None
```

```
    def optimise_weights(self, window_returns):
        mu = expected_returns.mean_historical_return(window_returns)
```

```

cov = risk_models.sample_cov(window_returns)
ef = EfficientFrontier(mu, cov)
weights = ef.max_sharpe(risk_free_rate=0.0)
cleaned_w = ef.clean_weights()
return cleaned_w

def apply_weights(self, start_idx, end_idx, weights):
    # Slice returns for the out-of-sample period
    period_returns = self.returns.iloc[start_idx:end_idx]
    # Compute portfolio returns (dot product each day)
    daily_returns = period_returns.mul(pd.Series(weights), axis=1).sum(axis=1)
    return daily_returns

def run(self):
    rebalanced_dates = self.returns.resample(self.freq).last().index
    all_returns = pd.Series(dtype=float)
    last_weights = None
    for i, rebalance_date in enumerate(rebalanced_dates):
        start_idx = max(0, self.returns.index.get_loc(rebalance_date) - self.lookback)
        window_returns =
self.returns.iloc[start_idx:self.returns.index.get_loc(rebalance_date)]
        if len(window_returns) < self.lookback//2: # insufficient data
            continue
        weights = self.optimise_weights(window_returns)
        self.weights_history.append((rebalance_date, weights))
        # Determine next rebalance date
        if i < len(rebalanced_dates) - 1:
            next_date = rebalanced_dates[i+1]
            end_idx = self.returns.index.get_loc(next_date)
        else:
            end_idx = len(self.returns)
        # Compute out-of-sample returns
        period_ret = self.apply_weights(self.returns.index.get_loc(rebalance_date),
end_idx, weights)
        # Apply transaction cost using turnover
        if last_weights is not None:
            turnover = 0.5 * sum(abs(weights.get(t, 0) - last_weights.get(t, 0)) for t in
set(weights) | set(last_weights))
            cost = turnover * self.cost_rate
            period_ret.iloc[0] -= cost # subtract cost on first day after rebalancing
        last_weights = weights
        all_returns = pd.concat([all_returns, period_ret])

```

```
self.portfolio_returns = all_returns
return all_returns
```

Key implementation choices:

- **Lookback window:** Use 252 trading days (≈ 1 year) as default. Longer windows (504 or 756 days) reduce estimation error but may be less responsive.
- **Rebalance frequency:** For swing trading, monthly or weekly rebalancing is practical.
- **Transaction cost rate:** A composite rate of 0.2–0.3% per trade approximates brokerage, STT, exchange charges, stamp duty and GST based on ClearTax figures **【660910205238419†L165-L246】**. Slippage and bid–ask spread can be modelled as an additional 0.02% (2 bps). Adjust according to broker.
- **Weight handling:** The apply_weights function multiplies daily returns by current weights; weights are held constant between rebalances.

2.4 Regime Filter Implementation

```
from hmmlearn.hmm import GaussianHMM
```

```
class VolatilityRegimeDetector:
```

```
    def __init__(self, returns, atr, n_states=2):
```

```
        self.returns = returns
```

```
        self.atr = atr
```

```
        self.n_states = n_states
```

```
        self.model = None
```

```
    def fit_hmm(self):
```

```
        # Fit an HMM on returns (assume Gaussian)
```

```
        X = self.returns.values.reshape(-1, 1)
```

```
        self.model = GaussianHMM(n_components=self.n_states, covariance_type='full')
```

```
        self.model.fit(X)
```

```
        return self.model
```

```
    def predict_regime(self):
```

```
        # Predict hidden states (0=low vol, 1=high vol)
```

```
        states = self.model.predict(self.returns.values.reshape(-1, 1))
```

```
        return pd.Series(states, index=self.returns.index)
```

```
    def simple_threshold(self, quantile=0.7):
```

```
        # Use ATR/price or realised volatility to classify regimes
```

```
        norm_vol = (self.atr /
```

```
self.returns.index.to_series().map(self.returns.abs().mean())).dropna()
```

```
        thresh = norm_vol.quantile(quantile)
```

```

regime = (norm_vol > thresh).astype(int) # 1=high volatility
return regime

```

class TrendRegimeDetector:

```

def __init__(self, price_series):

```

```

    self.price = price_series

```

```

def moving_average_regime(self, short=50, long=200):

```

```

    short_ma = self.price.rolling(short).mean()

```

```

    long_ma = self.price.rolling(long).mean()

```

```

    regime = (short_ma > long_ma).astype(int) # 1=uptrend, 0=downtrend

```

```

    return regime

```

Integration: The backtester can check the regime indicator at each rebalance date. If the volatility regime is high or the trend regime is down, the engine can reduce or suspend new positions (e.g., set weights to zero or allocate to cash). This regime filter helps avoid trading in unfavourable conditions **【654737399976859†L105-L116】** .

2.5 Benchmarking and Risk Decomposition

class Benchmarking:

```

def __init__(self, strategy_returns, tickers_returns, index_returns):

```

```

    self.strategy_returns = strategy_returns

```

```

    self.tickers_returns = tickers_returns

```

```

    self.index_returns = index_returns

```

```

def equal_weight_returns(self):

```

```

    eq_returns = self.tickers_returns.mean(axis=1)

```

```

    return eq_returns

```

```

def compute_alpha_beta(self):

```

```

    # Align strategy and index returns

```

```

    data = pd.concat([self.strategy_returns, self.index_returns], axis=1).dropna()

```

```

    Y = data.iloc[:, 0] # strategy

```

```

    X = add_constant(data.iloc[:, 1]) # index

```

```

    model = OLS(Y, X).fit()

```

```

    alpha = model.params['const']

```

```

    beta = model.params[data.columns[1]]

```

```

    pval_alpha = model.pvalues['const']

```

```

    return alpha, beta, pval_alpha

```

```

def information_ratio(self):

```

```

    active = self.strategy_returns - self.index_returns

```

```

    tracking_err = active.std()

```

```

    return active.mean() / tracking_err

```

Benchmark choices:

- Use **equal-weight** performance as a simple baseline.
- Use **^NSEI** (NIFTY 50) as the market benchmark.
- Compare alpha, beta and information ratio to judge whether the swing-trading portfolio adds value beyond passive exposure.

2.6 Metrics Engine and Validation Layer

class PerformanceMetrics:

```
def __init__(self, returns, freq=252):
```

```
    self.returns = returns
```

```
    self.freq = freq # number of trading days per year
```

```
    self.cum_returns = (1 + returns).cumprod()
```

```
def cagr(self):
```

```
    n_years = len(self.returns) / self.freq
```

```
    return (self.cum_returns.iloc[-1])** (1/n_years) - 1
```

```
def annualised_volatility(self):
```

```
    return self.returns.std() * np.sqrt(self.freq)
```

```
def sharpe_ratio(self, rf=0.0):
```

```
    excess = self.returns - rf/self.freq
```

```
    return excess.mean() / self.returns.std() * np.sqrt(self.freq)
```

```
def sortino_ratio(self, rf=0.0):
```

```
    downside = self.returns[self.returns < 0]
```

```
    downside_dev = downside.std() * np.sqrt(self.freq)
```

```
    excess = self.returns.mean() - rf/self.freq
```

```
    return excess / downside_dev
```

```
def max_drawdown(self):
```

```
    running_max = self.cum_returns.cummax()
```

```
    drawdown = (self.cum_returns - running_max) / running_max
```

```
    return drawdown.min()
```

```
def rolling_sharpe(self, window=126, rf=0.0):
```

```
    roll_mean = self.returns.rolling(window).mean()
```

```
    roll_std = self.returns.rolling(window).std()
```

```
    return ((roll_mean - rf/self.freq) / roll_std) * np.sqrt(self.freq)
```

```
def var_cvar(self, alpha=0.05):
```

```

sorted_returns = self.returns.sort_values()
idx = int(alpha * len(sorted_returns))
var = sorted_returns.iloc[idx]
cvar = sorted_returns.iloc[:idx].mean()
return var, cvar

```

class StrategyValidator:

```

def __init__(self, metrics, alpha_beta_pval, info_ratio):
    self.metrics = metrics
    self.alpha, self.beta, self.pval_alpha = alpha_beta_pval
    self.ir = info_ratio

def validate(self, sharpe_min=1.0, mdd_max=-0.2, ir_min=0.5):
    # Evaluate against thresholds
    sharpe = self.metrics.sharpe_ratio()
    mdd = self.metrics.max_drawdown()
    cagr = self.metrics.cagr()
    # Conditions
    cond_sharpe = sharpe >= sharpe_min
    cond_alpha = (self.pval_alpha < 0.05) and (self.alpha > 0)
    cond_mdd = mdd >= mdd_max # mdd is negative
    cond_ir = self.ir >= ir_min
    return {
        'Sharpe': sharpe,
        'CAGR': cagr,
        'MaxDrawdown': mdd,
        'Alpha': self.alpha,
        'Alpha_pval': self.pval_alpha,
        'Beta': self.beta,
        'InformationRatio': self.ir,
        'SharpeOK': cond_sharpe,
        'AlphaOK': cond_alpha,
        'DrawdownOK': cond_mdd,
        'IROK': cond_ir
    }

```

Validation rules:

- **Sharpe ratio** must exceed 1.0 to be acceptable **【288494383251952†L336-L347】** ; stronger strategies may target >2.0.
- **Positive, significant alpha** (p-value < 0.05). If alpha is insignificant, the strategy does not add value beyond market exposure.

- **Maximum drawdown** should not exceed -20% (adjust to risk appetite).
- **Information ratio** should exceed 0.5, indicating consistent outperformance.
- **Turnover** should remain manageable; apply cost constraints.

2.7 Practical Defaults and Real-World Considerations

Parameter	Typical default	Rationale
Lookback window	252 days ($\approx$ 1 year)	Balances responsiveness and estimation error. Longer windows (504 days) provide smoother estimates but adapt slowly to regime shifts.
Rebalance frequency	Monthly or weekly	Swing trading typically holds positions for days–weeks. Monthly rebalancing reduces turnover and costs; weekly can capture intermediate opportunities.
Transaction cost rate	$\sim 0.2\text{--}0.3\%$ per trade	Based on Indian costs: brokerage 0.05%, STT 0.1%, exchange charges 0.00325%, stamp duty 0.015%, GST 18% of fees 【660910205238419†L165-L246】. Add slippage (2 bps).
Risk-free rate	Use yield on 91-day Indian Treasury bills	Input for Sharpe/Sortino calculations.
Volatility threshold for regime	70th percentile of normalised ATR or realised volatility	Identifies high-volatility regimes 【953374386546441†L299-L316】.
Trend regime	50-day vs 200-day EMA crossover	A golden cross (short EMA > long EMA) indicates bullish regime 【868227748718115†L447-L487】.
Minimum Sharpe ratio	≥ 1.0 (target > 1.5)	Based on Wall Street Prep guidelines 【288494383251952†L336-L347】.
Alpha significance	p-value < 0.05	Statistical significance threshold for CAPM alpha.
Max drawdown	$\geq -20\%$	Ensures large losses are

Parameter	Typical default	Rationale
Information ratio	≥ 0.5	avoided; adjust for risk tolerance. Signifies consistent outperformance relative to benchmark.

2.8 Putting It All Together – Workflow Summary

1. **User Input:** The user enters a list of stock tickers (4–6). Choose a backtest period (e.g., last 10 years).
2. **Data Fetching:** DataFetcher downloads daily OHLCV data for the tickers and ^NSEI.
3. **Data Cleaning:** DataCleaner aligns dates, fills missing values, computes returns, ATR and volatility.
4. **Walk-Forward Optimisation:**
 - For each rebalance date: define an **in-sample** window (lookback) and compute expected returns and covariance using FeatureStore.
 - Optimise portfolio weights via MaxSharpeOptimizer (using PyPortfolioOpt).
 - Apply weights to the **out-of-sample** period until next rebalance; adjust the first day for transaction costs based on turnover.
 - Record weights and compute daily portfolio returns.
5. **Regime Filter:** At each rebalance, evaluate volatility and trend regimes. If high-volatility or bearish regime, reduce or suspend trading.
6. **Benchmark Calculation:** Compute equal-weight returns and ^NSEI returns.
7. **Performance Evaluation:** Use PerformanceMetrics to compute CAGR, Sharpe, Sortino, max drawdown, VaR/CVaR and turnover.
8. **Risk Decomposition:** Run CAPM regression using Benchmarking to obtain alpha, beta and information ratio.
9. **Validation:** Use StrategyValidator to check whether metrics meet pre-defined thresholds.

10. **Dashboard Output:** Present summary statistics, equity curves, rolling metrics and regime labels in the Streamlit tab. Allow the user to adjust parameters (lookback, rebalancing, cost assumptions) and observe changes in performance.

Conclusion

A robust backtesting engine for swing trading must combine **sound statistical foundations**, **rigorous bias mitigation**, and **practical implementation** that handles real-world constraints like transaction costs and market regimes. The blueprint above outlines a modular system where each component—from data ingestion to optimisation, regime detection, benchmarking, and validation—can be developed independently and integrated into a dashboard. Using **walk-forward optimisation** ensures that weight estimation remains adaptive while minimising overfitting [\[830181354955615†L117-L133\]](#) [\[830181354955615†L161-L182\]](#) . Incorporating **risk-adjusted metrics** (Sharpe, Sortino, alpha, information ratio) and **regime filters** (volatility and trend) improves reliability across market conditions [\[953374386546441†L299-L316\]](#) [\[868227748718115†L447-L487\]](#) . Finally, explicit **cost modelling** based on Indian trading charges [\[660910205238419†L165-L246\]](#) and **validation rules** help ensure that the resulting strategies are both realistic and actionable for live trading.